

EXPERIMENT – 9

Title

Organizing Kubernetes Pods Using Labels and Namespaces

Aim

To organize and manage Kubernetes Pods efficiently using **namespaces** and **labels** by creating separate environments for development and production.

Description

This experiment demonstrates how Kubernetes namespaces are used to logically separate environments such as development (dev) and production (prod). Labels are applied to Pods to categorize them based on their role, such as frontend and backend, enabling easy filtering and management of resources.

Project Structure

Program-9/
├── namespaces.yaml
└── pods.yaml

Requirements

- Kubernetes cluster (Minikube)
 - kubectl command-line tool
 - Basic knowledge of YAML
 - Docker images (nginx)
-

Theory

A **Namespace** in Kubernetes provides a mechanism for isolating groups of resources within a single cluster. It helps in organizing resources for different environments or teams.

Labels are key–value pairs attached to Kubernetes objects. They are used to identify, group, and filter resources such as Pods.

Using namespaces and labels together improves resource management, avoids naming conflicts, and simplifies monitoring.

Procedure

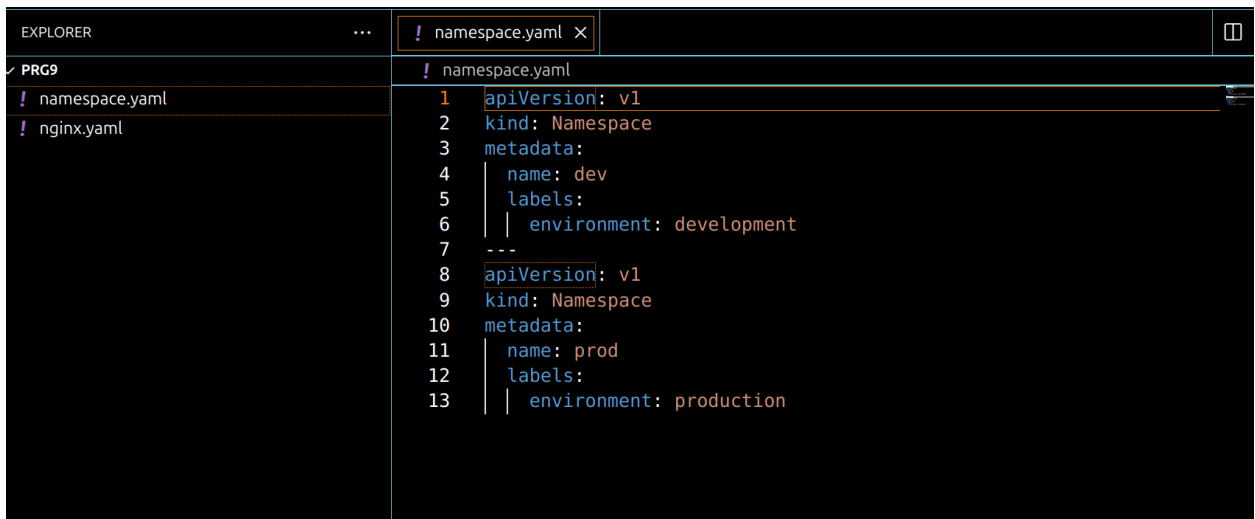
Step 1: Create Namespace YAML (**namespaces.yaml**)

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
```

Apply the namespaces:

```
kubectl apply -f namespaces.yaml
```



```
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: dev
5    labels:
6      environment: development
7  ---
8  apiVersion: v1
9  kind: Namespace
10 metadata:
11   name: prod
12   labels:
13     environment: production
```

Step 2: Create Pod YAML (**pods.yaml**)

Frontend Pod (dev namespace)

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: dev
  labels:
    app: frontend
    env: dev
spec:
  containers:
    - name: frontend
      image: nginx
      ports:
        - containerPort: 80
```

Backend Pod (prod namespace)

```
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: prod
  labels:
    app: backend
    env: prod
spec:
  containers:
    - name: backend
      image: nginx
      ports:
        - containerPort: 80
```

Apply the pods:

```
kubectl apply -f pods.yaml
```

```
EXPLORER ... ! namespace.yaml ! nginx.yaml x
v PRG9
! namespace.yaml
! nginx.yaml

! nginx.yaml
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: frontend
5   namespace: dev
6   labels:
7     app: my-frontend
8     env: dev
9 spec:
10  containers:
11    - name: frontend
12      image: nginx:latest
13      ports:
14        - containerPort: 80
15  ---
16 apiVersion: v1
17 kind: Pod
18 metadata:
19   name: backend
20   namespace: prod
21   labels:
22     app: my-backend
23     env: prod
24 spec:
25  containers:
26    - name: backend
27      image: nginx:latest
28      ports:
29        - containerPort: 80
30
```

Step 3: Verify Pods by Namespace

```
kubectl get pods -n dev
kubectl get pods -n prod
```

Step 4: Filter Pods by Label

List frontend pods in dev namespace:

```
kubectl get pods -n dev -l app=frontend
```

List backend pods in prod namespace:

```
kubectll get pods -n prod -l app=backend
```

Step 5: Cleanup Process

Delete all pods:

```
kubectll delete pods --all -n dev  
kubectll delete pods --all -n prod
```

Delete all services:

```
kubectll delete svc --all
```

Output

Pods are successfully created in separate namespaces and can be filtered based on labels.

Result

Kubernetes Pods were successfully organized using namespaces for environment separation and labels for role-based identification.
